

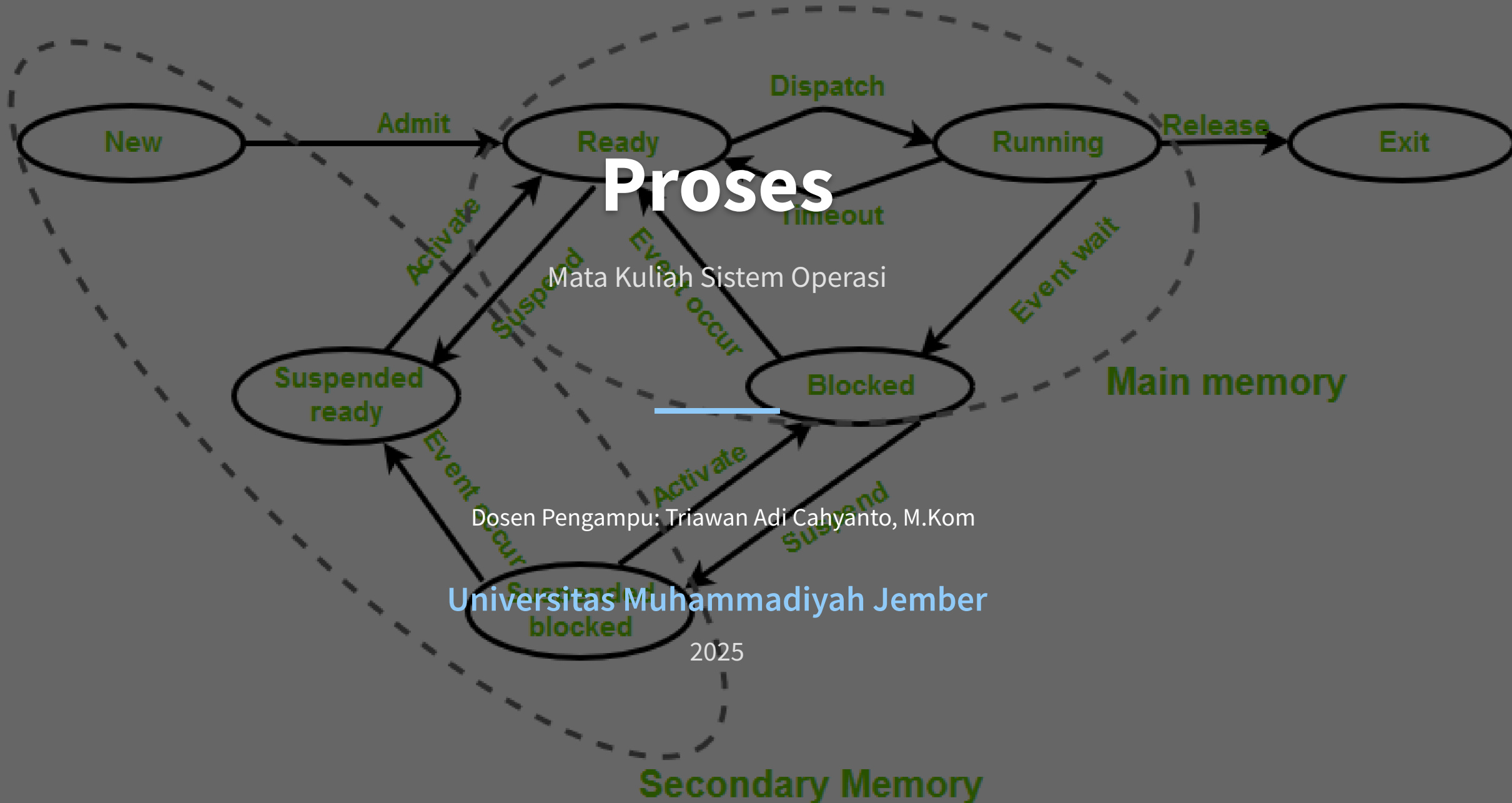
Proses

Mata Kuliah Sistem Operasi

Dosen Pengampu: Triawan Adi Cahyanto, M.Kom

Universitas Muhammadiyah Jember

2025



Process Concept

Konsep Dasar Proses dalam Sistem Operasi

i Definisi & Perbedaan

- **Proses** = program yang sedang dieksekusi
- **Program** = entitas pasif (file di disk)
- **Proses** = entitas aktif (sedang berjalan)

🔧 Komponen Proses

Text Section: Instruksi yang dieksekusi (read-only)

Data Section: Variabel global

Heap Section: Memori dialokasikan dinamis

Stack Section: Data sementara (parameter fungsi, alamat return, variabel lokal)

⚙️ Process Control Block (PCB)

🌀 Process ID (PID)

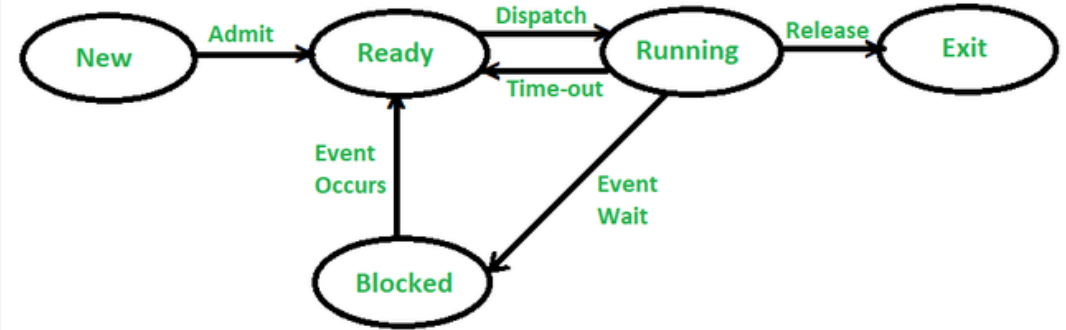
🔄 Process State

📈 Priority

📁 I/O Information

📄 File Descriptors

📊 Accounting Info



↻ State Proses

New

Ready

Running

Waiting

Terminated

→ Transisi State Proses

- **New → Ready**: Proses dimuat ke memori
- **Ready → Running**: Proses dipilih untuk dieksekusi
- **Running → Waiting**: Proses menunggu I/O
- **Waiting → Ready**: I/O selesai
- **Running → Ready**: Time slice habis
- **Running → Terminated**: Proses selesai

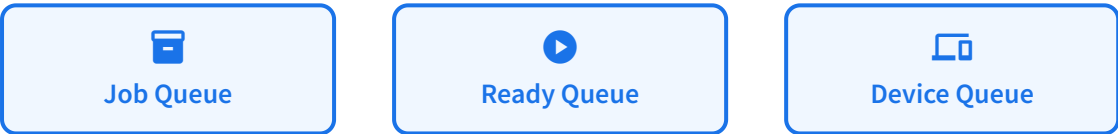
Process Scheduling

Penjadwalan Proses dalam Sistem Operasi

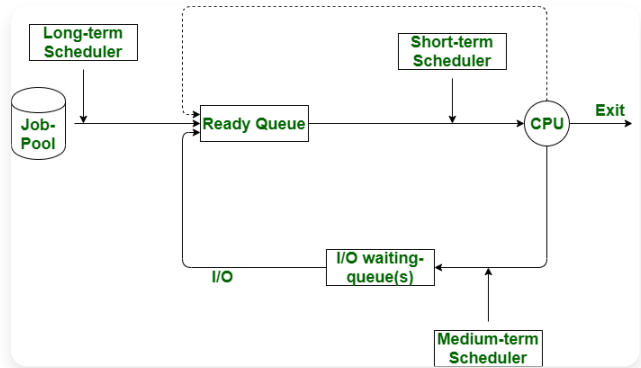
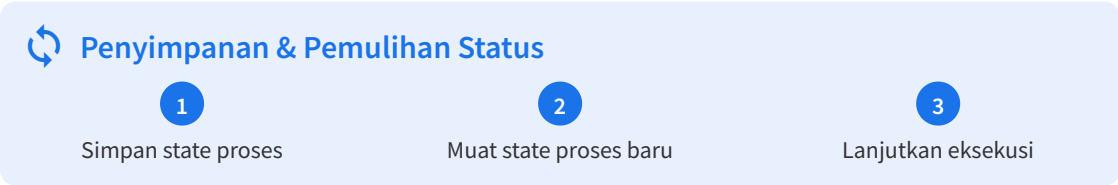
🕒 Tujuan Penjadwalan

- Maksimalkan CPU
- Efisiensi sistem
- Waktu tunggu minimal
- Respons cepat

📁 Jenis Antrian



↔ Context Switch



📁 Jenis Penjadwalan

- 🕒 Long-term (Job Scheduler)
Memilih proses dari disk ke memori
- 📁 Short-term (CPU Scheduler)
Memilih proses dari ready queue
- ↕ Medium-term (Swapper)
Memindahkan proses dari memori ke disk

📁 Kategori Penjadwalan

Non-Preemptive	Preemptive
Proses berjalan hingga selesai	Proses dapat diinterupsi
Lebih sederhana	Respons lebih cepat

Operations on Processes

Operasi yang Dapat Dilakukan pada Proses

⚙️ Operasi pada Proses

+ Creation

Pembuatan proses baru dengan `fork()`

🕒 Scheduling

Perubahan state dari ready ke running

🚫 Blocking

Proses masuk ke state waiting karena I/O

↔️ Preemption

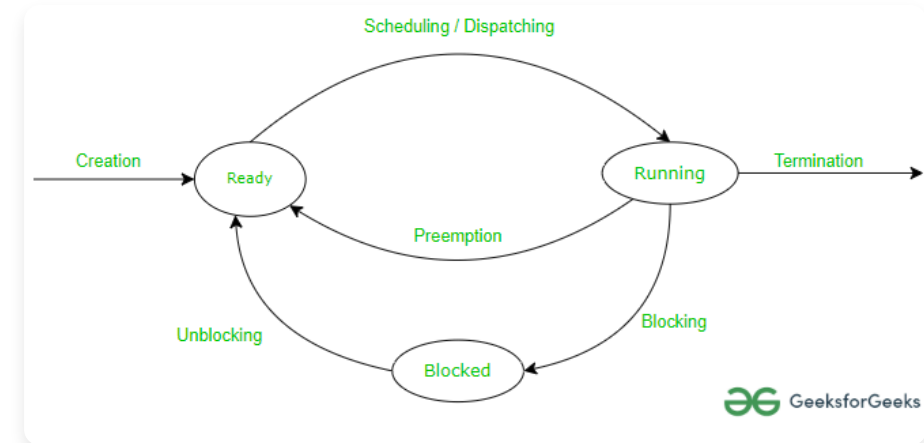
Proses dipindahkan dari running ke ready

✖️ Process Termination

- **Normal**: Proses selesai eksekusi
- **Error**: OS menghentikan proses
- **Fatal Error**: Masalah hardware

<> Process Creation Example

```
pid = fork();
if (pid == 0) {
    // Child process code
} else {
    // Parent process code
}
```



🏠 Hierarki Proses

- **Parent-Child Relationship**
- Proses anak mewarisi **sumber daya** dari induk
- Proses induk dapat **menunggu** proses anak

⚠️ Orphan & Zombie Processes

👤 Orphan Process

Proses anak yang induknya sudah berakhir

👤 Zombie Process

Proses selesai tapi masih ada di tabel proses

Poin Penting

- Operasi proses **krusial** untuk multitasking
- Setiap operasi mempengaruhi **state proses**
- Manajemen proses yang baik **mengoptimalkan** sumber daya

Interprocess Communication

Komunikasi Antar Proses dalam Sistem Operasi

↔ Konsep Dasar IPC

- IPC adalah **mekanisme komunikasi antar proses**
- **Tujuan utama**: pertukaran data antar proses
- Membantu proses **sinkronisasi aktivitas** dan berbagi informasi
- Mencegah **konflik akses** ke sumber daya bersama

🔗 Sinkronisasi & Koordinasi

- Dibutuhkan untuk **menghindari race condition**
- Menjaga **konsistensi data** saat diakses bersama
- Mengatur **urutan eksekusi** proses yang benar



Semaphore

Mengontrol akses ke sumber daya bersama



Monitor

Struktur data dengan operasi atomik

Pentingnya IPC

- **Distribusi sumber daya** - berbagi sumber daya antar proses
- **Modularitas** - memecah aplikasi besar menjadi proses kecil
- **Efisiensi** - mengoptimalkan penggunaan CPU dan memori

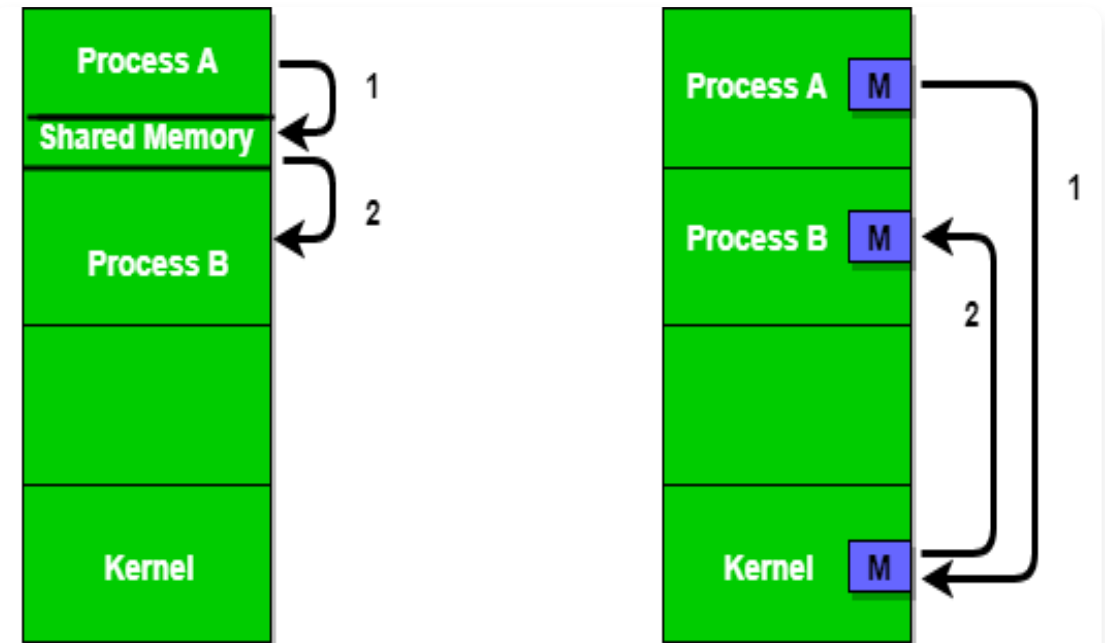


Figure 1 - Shared Memory and Message Passing

🔗 Model Fundamental IPC

⚙ Shared Memory

Proses mengakses area memori yang sama secara langsung

- ✓ Kecepatan tinggi
- 🔗 Perlu sinkronisasi
- ⚠ Risiko keamanan

💬 Message Passing

Proses bertukar pesan melalui sistem operasi

- ✓ Aman dan terstruktur
- ⌚ Lebih lambat
- 🌐 Cocok untuk sistem terdistribusi

IPC in Shared-Memory Systems

Komunikasi Antar Proses Menggunakan Memori Bersama

Konsep Shared Memory

- Area memori bersama digunakan oleh beberapa proses
- Mekanisme IPC paling cepat karena akses langsung
- Tidak memerlukan intervensi kernel setelah memori dibagikan

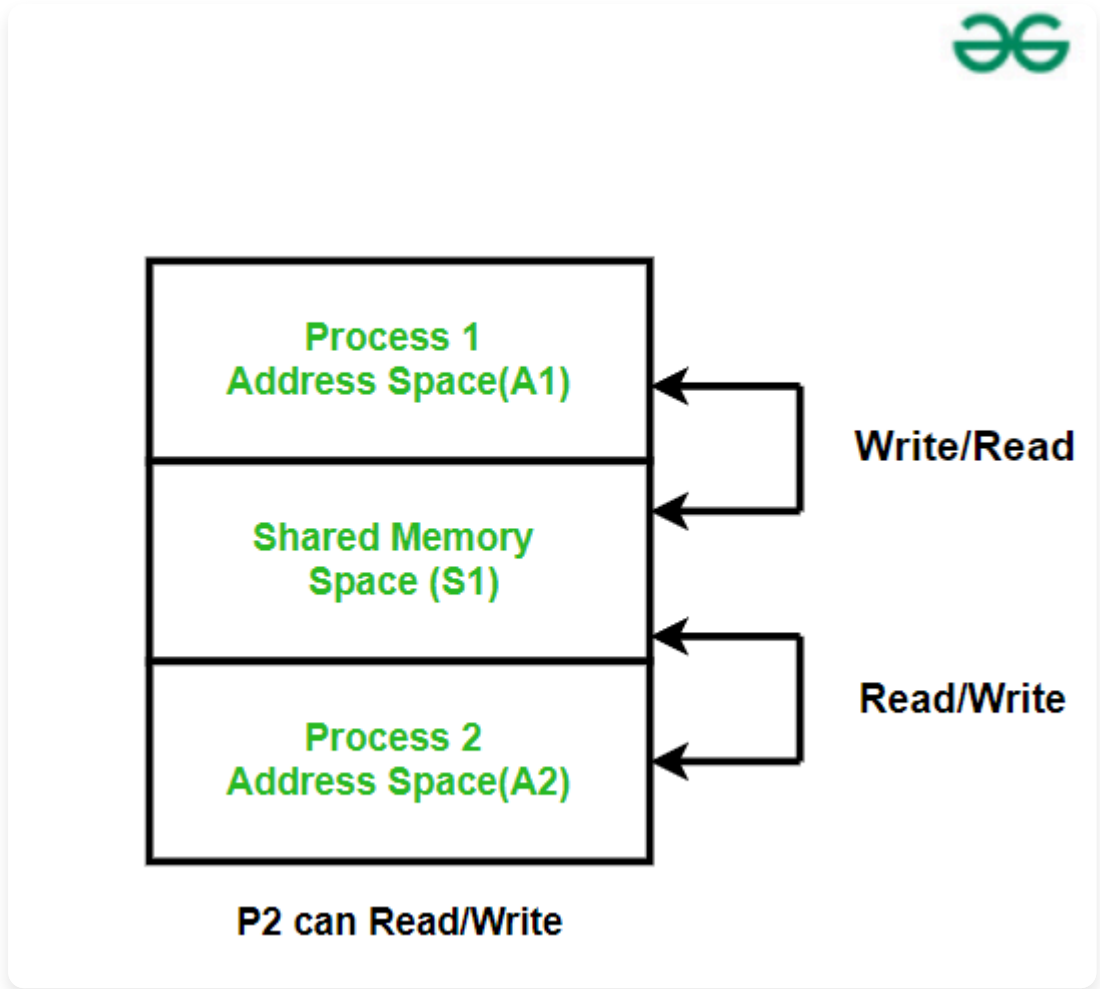
Cara Kerja Shared Memory IPC

- 1

Pembuatan Shared Memory
Proses induk membuat segmen memori bersama
- 2

Koneksi ke Shared Memory
Proses menghubungkan diri ke segmen memori
- 3

Sinkronisasi
Mekanisme sinkronisasi mencegah race condition



Keuntungan & Kerugian

Keunggulan

- **Cepat** - akses langsung ke memori
- **Efisien** untuk transfer data besar
- **Overhead rendah** - tidak perlu kernel

Kelemahan

- **Sinkronisasi kompleks** - perlu mekanisme khusus
- **Risiko keamanan** - akses tidak terbatas
- **Pembersihan manual** - perlu detach & remove

<> System Calls yang Digunakan

ftok()

Menghasilkan kunci unik

shmget()

Membuat segmen shared memory

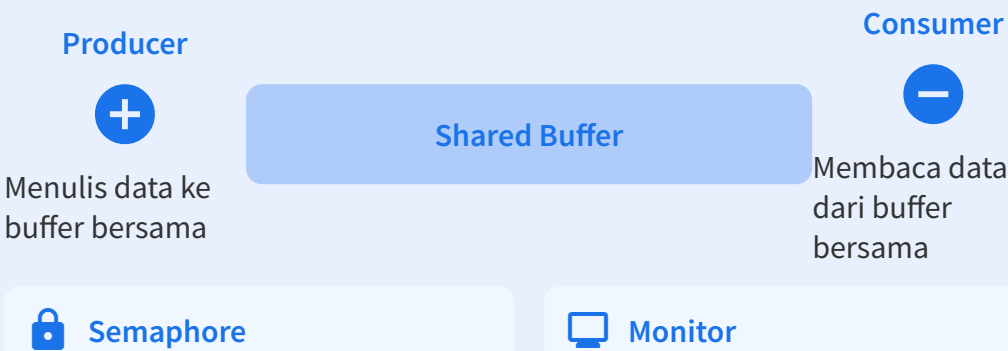
shmat()

Menghubungkan proses ke shared memory

shmdt()

Memutus koneksi dari shared memory

Contoh: Producer-Consumer Problem



IPC in Message-Passing Systems

Komunikasi Antar Proses Menggunakan Pesan

📋 Konsep Message Passing

- Komunikasi melalui **pesan**, bukan berbagi memori
- Proses **tidak berbagi memori** secara langsung
- Komunikasi melalui operasi **send()** dan **receive()**

⚙️ Cara Kerja Message Passing IPC

1 Pengiriman Pesan

Proses pengirim memanggil operasi send() ke kernel

2 Pengiriman oleh Kernel

Kernel mengirimkan pesan ke proses penerima

3 Penerimaan Pesan

Proses penerima memanggil operasi receive()

↔️ Metode Komunikasi

🔗 Direct

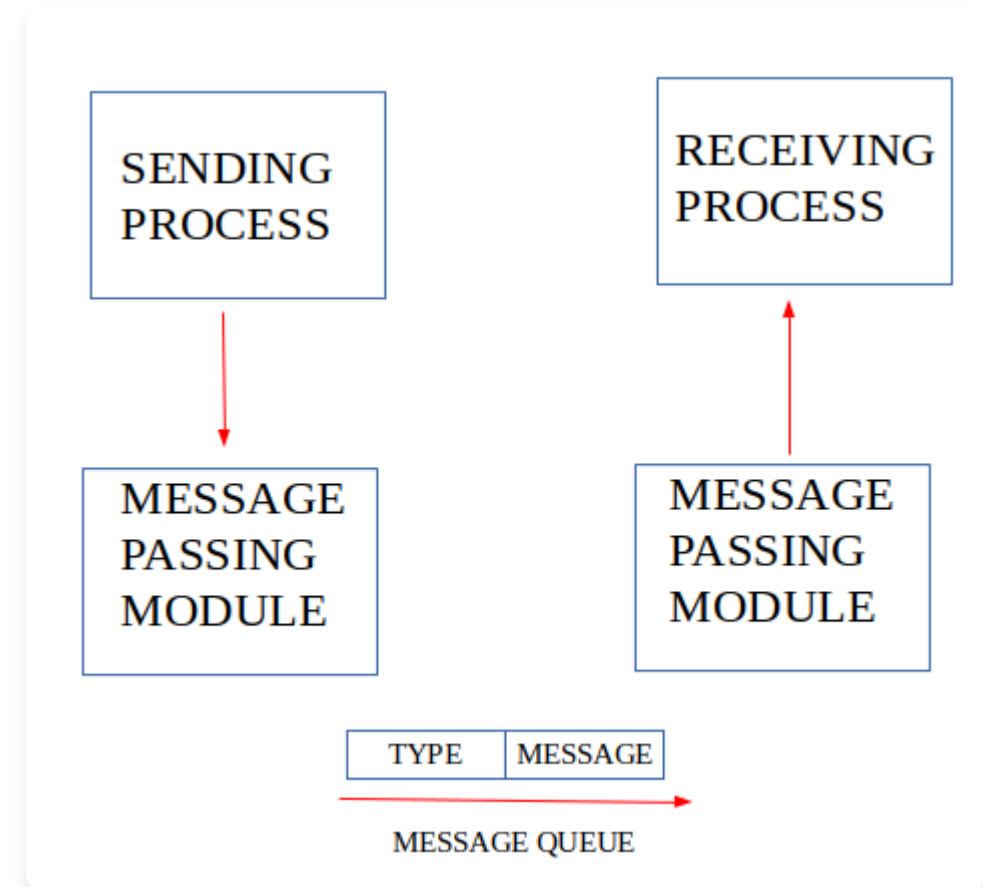
Komunikasi langsung antar proses

- ✓ Proses mengetahui identitas penerima
- ✓ send(receiver, message)

✉️ Indirect

Komunikasi melalui mailbox/port

- ✓ Proses tidak mengetahui identitas penerima
- ✓ send(mailbox, message)



🔄 Sifat Komunikasi



Synchronous

Blocking - proses menunggu hingga pesan diterima



Asynchronous

Non-blocking - proses melanjutkan eksekusi

📊 Keuntungan & Kerugian

👍 Keuntungan

- **Sederhana** - tidak perlu manajemen memori
- **Aman** - tidak ada akses langsung ke memori
- **Terdistribusi** - cocok untuk sistem jaringan

👎 Kerugian

- **Lebih lambat** - melibatkan kernel
- **Overhead tinggi** - setiap pesan memerlukan interaksi
- **Konsumsi sumber daya** - memori untuk buffer pesan

Examples of IPC Systems

Berbagai Implementasi Komunikasi Antar Proses

Jenis-Jenis Sistem IPC

- Dikategorikan berdasarkan mekanisme komunikasi
- Pemilihan bergantung pada kebutuhan aplikasi

Pipes

Saluran komunikasi satu arah antar proses terkait

- Anonymous & Named Pipes

Sockets

Komunikasi antar proses pada host berbeda

- TCP & UDP Protocols

Shared Memory

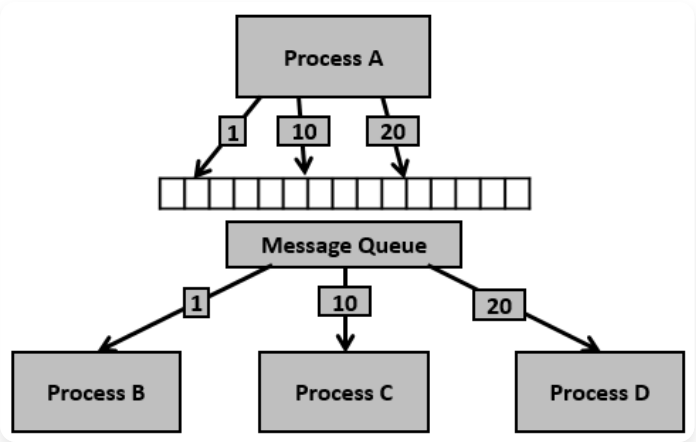
Area memori yang dapat diakses beberapa proses

- Cepat untuk data besar

Semaphores

Mekanisme kontrol akses ke sumber daya bersama

- Mencegah race condition



Implementasi IPC Nyata

POSIX Shared Memory

Implementasi standar di Unix/Linux

`shm_open()` `mmap()`

Windows Named Pipes

Komunikasi antar proses di Windows

`CreateNamedPipe()` `ConnectNamedPipe()`

Message Queues

Antrian pesan yang dikelola oleh kernel

`msgget()` `msgsnd()` `msgrcv()`

Pemilihan Sistem IPC

Kebutuhan	Sistem IPC yang Cocok
Proses terkait, data kecil	Pipes
Komunikasi jaringan	Sockets
Data besar, performa tinggi	Shared Memory
Kontrol akses sumber daya	Semaphores

Communication in Client-Server Systems

Komunikasi dalam Sistem Client-Server

↔ Konsep Komunikasi Client-Server

- **Arsitektur Client-Server**: komunikasi antar proses di jaringan
- Client mengirim **permintaan (request)** ke server
- Server memproses dan mengirimkan **respons**
- Widely used in **distributed systems** dan **internet applications**

↔ Metode Komunikasi

🔌 Socket Programming

Endpoint komunikasi untuk transfer data dua arah

☎ RPC

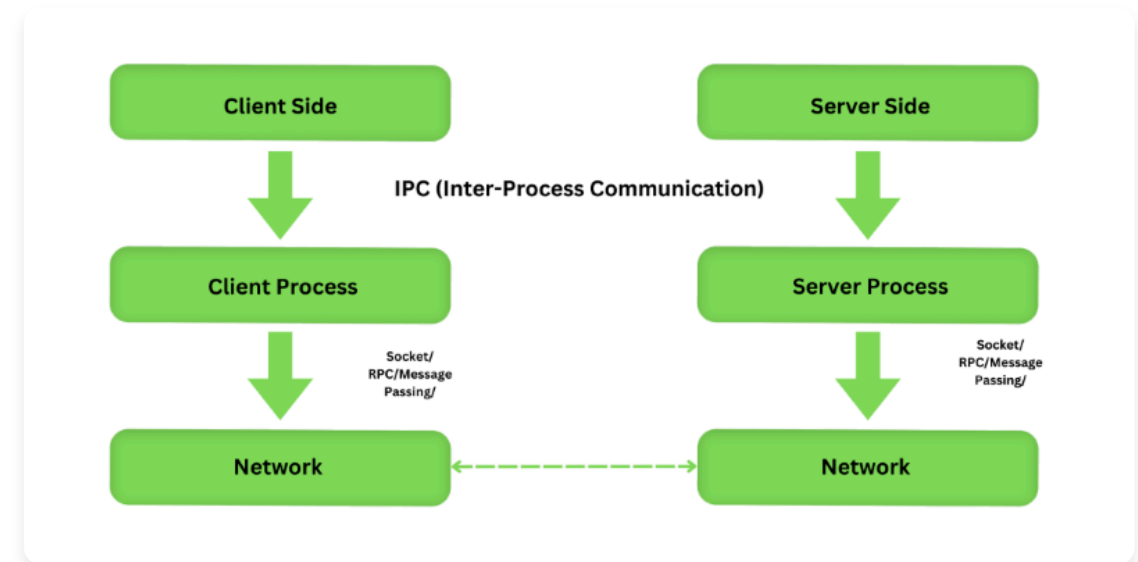
Remote Procedure Call - eksekusi fungsi di server

<> RMI

Remote Method Invocation - khusus Java

📁 Distributed File Systems

Akses file dari mesin jarak jauh



🚩 Protokol Umum



TCP/IP

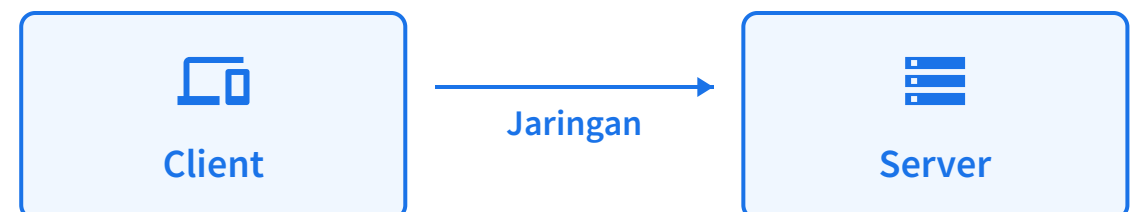
Protokol dasar komunikasi internet dengan koneksi yang andal



HTTP/HTTPS

Protokol untuk transfer data web antara client dan server

🏠 Arsitektur Client-Server



Summary

Rangkuman Materi Processes dalam Sistem Operasi



Process Concept

- ✓ Program dalam eksekusi
- ✓ Struktur memori: Text, Data, Heap, Stack
- ✓ State: New, Ready, Running, Waiting, Terminated
- ✓ PCB: informasi status proses



Process Scheduling

- ✓ Tujuan: maksimalkan penggunaan CPU
- ✓ Antrian: Job, Ready, Device
- ✓ Penjadwalan: Long-term, Short-term, Medium-term
- ✓ Context switch: penyimpanan & pemulihan status



Operations on Processes

- ✓ Creation: fork()
- ✓ Hierarki: parent-child relationship
- ✓ Terminasi: normal atau error
- ✓ Orphan & Zombie processes



Interprocess Communication

- ✓ Tujuan: pertukaran data antar proses
- ✓ Model: Shared Memory, Message Passing
- ✓ Sinkronisasi untuk menghindari race condition



IPC Systems

- ✓ Shared Memory: cepat, efisien
- ✓ Message Passing: Direct, Indirect
- ✓ Implementasi: POSIX, Pipes, Sockets
- ✓ Client-Server: Socket, RPC, RMI



Client-Server Systems

- ✓ Arsitektur komunikasi antar proses di jaringan
- ✓ Metode: Socket Programming, RPC, RMI
- ✓ Protokol: TCP/IP, HTTP



Kesimpulan: Proses adalah unit fundamental eksekusi dalam sistem operasi. Manajemen proses yang efektif meliputi penjadwalan, operasi, dan komunikasi antar proses untuk memungkinkan multitasking dan optimalisasi sumber daya. IPC memungkinkan proses untuk berkoordinasi dan berbagi informasi, yang sangat penting dalam sistem modern yang kompleks.